

Practical - 2

Aim: To Implementation and Time analysis of linear and binary search algorithm.

1) Linear Search

Algorithm:

Input : An array $A[0 \dots n-1]$ and a search element key
Output : position of key if found, otherwise "element not found"

Step 1 :- Read array A , n and key
Step 2 :- Start from the first element of the array
Step 3 :- Compare $A[i]$ with key
Step 4 :- If $A[i] == \text{key}$, print the position
Step 5 :- If $A[i] \neq \text{key}$, move to the next element
Step 6 :- Repeat the comparison until the key is found or the array ends
Step 7 :- If key is not found, print "Element not found"
Step 8 :- End

Program:

```
#include <iostream>
using namespace std;
class LinearSearch{
public:
    int arr[100], n, key;
    void arrayInput()
    {
        cout << "Enter number of elements: ";
        cin >> n;
        cout << "Enter the elements: ";
        for (int i = 0; i < n; i++){
            cin >> arr[i];
        }
        cout << "Enter the element to search: ";
        cin >> key;
    }
};
```

Name: Chinni Thabitha
Enrollment Number: 92460118038

```
void printArray()
{
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
}

void search()
{
    int i;

    for (i = 0; i < n; i++)
    {
        if (arr[i] == key)
        {
            cout << "Element found at index " << i << endl;
            break;
        }
    }

    if (i == n)
    {
        cout << "Element not found." << endl;
    }
}

};

int main()
{
    LinearSearch ls;

    ls.arrayInput();

    cout << "\nArray: ";
    ls.printArray();

    ls.search();

    return 0;
}
```

Output:

```
PS C:\MinGW\bin> & "C:\Users\chinn\Downloads\dsa in c++\linear search.exe"
Enter number of elements: 6
Enter the elements: 12
6
8
2
21
1
Enter the element to search: 8

Array: 12 6 8 2 21 1
Element found at index 2
```

Time Complexity:

for ($i=0; i < n; i++$) {
 if ($arr[i] == key$)
 break;
}

Best case: if element is found at the first position

$i=0 \rightarrow 1$
 $i < n \rightarrow 1$
 $arr[i] == key \rightarrow 1$
 $T(n) = 1 + 1 + 1$
 $= 3$
 $O(1)$
Best case = $O(1)$

Average case: element is found in the middle

$i=0 \rightarrow 1$
 $i < n \rightarrow \frac{n}{2} + 1$
 $i++ \rightarrow \frac{n}{2}$
 $arr[i] == key \rightarrow \frac{n}{2}$
 $T(n) = 1 + \frac{n}{2} + 1 + \frac{n}{2} + \frac{n}{2}$
 $= 3\frac{n}{2} + 2$
 $= O(n)$
Average case = $O(n)$

Worst case: element is at the last or is not present

$i < n \rightarrow n + 1$
 $i++ \rightarrow n$
 $arr[i] == key \rightarrow n$
 $T(n) = 1 + n + 1 + n + n$
 $= 3n + 2$
 $= O(n)$
Worst case = $O(n)$

2) Binary Search:

Algorithm:

Input: A sorted array $A[0 \dots n-1]$ and a search element key
Output: Position of key if found, otherwise "element not found".

Step 1:- Read array A, n and key
Step 2:- set $low = 0$ and $high = n - 1$
Step 3:- find the middle element
 $mid = \frac{low + high}{2}$
Step 4:- compare $A[mid]$ with key
Step 5:- If $A[mid] == key$, print the position
Step 6:- If $key < A[mid]$, search in the left half
Step 7:- If $key > A[mid]$, search in the right half
Step 8:- Repeat the above steps until the key is found or $low > high$
Step 9:- If key is not found, print "element not found"
Step 10:- End

Program:

```
#include <iostream>
using namespace std;
class BinarySearch{
public:
    int arr[100], n, key;
    void arrayInput(){
        cout << "Enter number of elements: ";
        cin >> n;
        cout << "Enter the sorted elements: ";
        for (int i = 0; i < n; i++){
            cin >> arr[i];
        }
        cout << "Enter the element to search: ";
        cin >> key;
    }
    void printArray(){
```

```
for (int i = 0; i < n; i++)
{
    cout << arr[i] << " ";
}
cout << endl;
}

void search()
{
    int low = 0;
    int high = n - 1;

    while (low <= high)
    {
        int mid = (low + high) / 2;

        if (arr[mid] == key)
        {
            cout << "Element found at index " << mid << endl;
            return;
        }
        else if (key < arr[mid])
        {
            high = mid - 1;
        }
        else
        {
            low = mid + 1;
        }
    }

    cout << "Element not found." << endl;
}

};

int main()
{
    BinarySearch bs;
    bs.arrayInput();
    cout << "\nArray: ";
    bs.printArray();
    bs.search();
    return 0;
}
```

Output:

```
PS C:\MinGW\bin> & "C:\Users\chinn\Downloads\dsa in c++\binarysearch.exe"
Enter number of elements: 6
Enter the sorted elements: 6
7
8
9
12
22
Enter the element to search: 9

Array: 6 7 8 9 12 22
Element found at index 3
```

Time complexity:

Binary search (A[], n, key) { $\rightarrow T(n)$

 low = 0; $\rightarrow (1)$

 high = n - 1; $\rightarrow (1)$

 while (low <= high) { $\rightarrow (1)$

 mid = $\frac{low + high}{2}$; $\rightarrow T(\frac{n}{2})$

 if (A[mid] == key) { $\rightarrow (1)$

 return mid;

 } else if (key < A[mid]) { $\rightarrow (1)$

 high = mid - 1;

 } else { $\rightarrow (1)$

 low = mid + 1;

 }

 } return -1

 }

$T(n) = T(\frac{n}{2}) + 1$

$T(n) = \begin{cases} 1 & n = 1 \\ T(\frac{n}{2}) + 1 & n > 1 \end{cases}$

$$\begin{aligned}
 T(n) &= T\left(\frac{n}{2}\right) + 1 \\
 T(n) &= T\left(\frac{n}{2}\right) + 1 \\
 &= T\left(\frac{n}{4}\right) + 1 + 1 \\
 &= T\left(\frac{n}{4}\right) + 2 \\
 T(n) &= T\left(\frac{n}{4}\right) + 2 \\
 &= T\left(\frac{n}{8}\right) + 2 + 1 \\
 &= T\left(\frac{n}{8}\right) + 3 \\
 T(n) &= T\left(\frac{n}{2^k}\right) + 3 \\
 &= T\left(\frac{n}{2^k}\right) + k \rightarrow T\left(\frac{n}{n}\right) + \log_2 n \\
 &= T(1) + \log_2 n \\
 &= 1 + \log_2 n \\
 n &= 1 \\
 \frac{n}{2^k} &= 1 \\
 n &= 2^k \\
 \log n &= \log 2^k \\
 \log n &= k \log 2 \\
 k &= \frac{\log n}{\log 2} \\
 k &= \log_2 n \\
 T(n) &= O(\log n) \rightarrow \text{in worst case} \\
 \text{Best case} &\rightarrow O(1) \\
 \text{Average case} &\rightarrow O(\log n)
 \end{aligned}$$

Conclusion:

The implementation and time analysis of Linear Search and Binary Search helps us understand their efficiency. Linear Search is simple and can work on any array, but it may require $O(n)$ comparisons. Binary Search is faster with $O(\log n)$ time in average and worst cases, but it requires the array to be sorted. Therefore, Binary Search is more efficient for searching large sorted dataset.